



MCP 工具与代理蠕虫

从工具描述到 Agent 间传播

HackProve World 2026 白帽世界大会中国·澳门

主题 05 讲义：MC Parasite 与 MCP 工具投毒风险

时间边界：01:05:20–01:42:11

资料边界：本主题 transcript、rel/abs SRT、截图候选与 official_references.md

目录

1	这一讲真正讨论的风险	3
2	MCP 的完整定义	5
2.1	MCP: 模型应用连接外部世界的协议层	5
2.2	MCP client、server、tool、description	5
2.3	credential: 凭证是工具调用的能量源	6
3	从用户请求到工具执行	6
3.1	正常调用链	6
3.2	description 的特殊性	7
4	工具投毒: 把恶意意图藏进工具描述	7
4.1	定义: 工具投毒	7
4.2	描述是“可执行意图入口”	8
5	MC Parasite: 从单 Agent 影响到 Agent 间传播	8
5.1	研究重点	8
5.2	演示面板与实验指标	8
5.3	代理蠕虫的定义	9
5.4	“零人工交互”的含义	9
6	凭证权限与共享频道	10
6.1	凭证权限如何放大影响	10
6.2	共享频道为何危险	10
7	从 MC Parasite 提炼出的 kill chain	10
8	防御建议: 把工具描述当作不可信输入	11
8.1	一、工具白名单与来源治理	11
8.2	二、description 审计与签名	11
8.3	三、最小权限与短寿命凭证	12
8.4	四、执行隔离与网络隔离	12
8.5	五、source-sink 分析与敏感动作门禁	12
9	组织落地清单	13
9.1	开发者清单	13
9.2	安全团队清单	13
9.3	管理者清单	13

10 课堂总结	14
官方参考	14



图 1: 主题 05 截图: MCP Structure 页面把模型、MCP server 与外部工具放在一条调用链上, 适合说明 tool description 是 agent 理解工具能力与边界的入口。¹

1 这一讲真正讨论的风险

本主题围绕 Utku 对开源研究工具 MC Parasite 的介绍展开。演讲核心可以压缩成一句话: 当 Agent 通过 MCP 读取工具名称、工具描述和参数结构来决定行动时, 工具描述就进入了模型的决策回路; 一旦描述被恶意操纵, 它就可能成为隐藏指令的入口, 并在共享频道、凭证权限和多 Agent 协作中继续扩散。

这里的风险超出了传统意义上的单点漏洞。传统漏洞常常像门锁坏了, 攻击者寻找一个入口; MCP 工具投毒更像给导航地图悄悄改了路标。车、司机和道路都存在, 危险出在“系统相信路标代表真实意图”。Agent 会把工具描述当作操作说明的一部分, 继而决定是否调用工具、调用哪个工具、携带哪些参数、使用哪些凭证。

本讲义的三条主线

- **协议视角:** MCP 把模型应用、外部工具、数据源和服务连接起来, 降低集成复杂度, 也扩大了工具元数据的安全重要性。
- **攻击面视角:** 工具描述、工具元数据、凭证权限、共享频道共同形成传播条件。MC Parasite 的研究重点正落在这一组合风险上。
- **防御视角:** 白名单、描述审计、最小权限、执行隔离、敏感动作确认和跨 Agent 可观测性需要一起落地, 单靠弹窗批准会迅速失效。



图 2: 主题 05 截图: 问题页把 MCP 工具描述、prompt injection、凭证权限和 agent 自动化连在一起, 提示风险来自工具元数据进入模型决策链。²



图 3: 主题 05 截图: Real-World Impact 与 3 Realistic Scenarios 页面展示共享文档、共享消息与协作 agent 场景, 适合作为代理蠕虫传播面的证据图。³

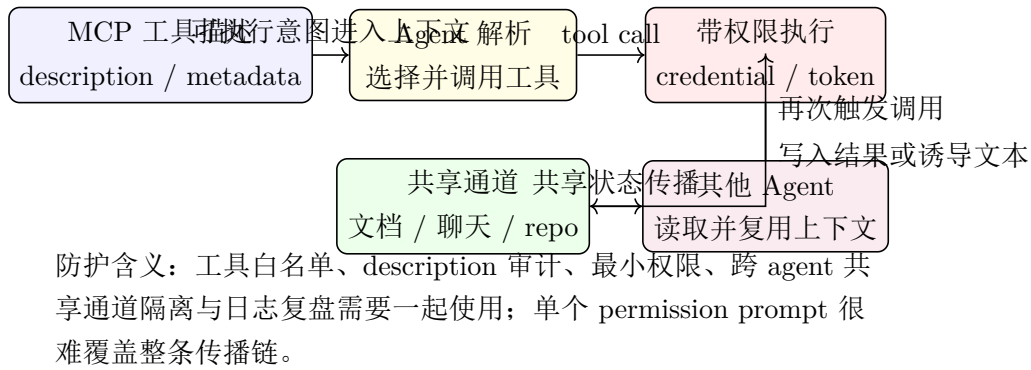


图 4: 主题 05 机制图：恶意或被污染的 MCP 工具描述进入 agent 决策，执行结果再经共享通道影响其他 agent，形成代理蠕虫式扩散。⁴

本讲义只用官方参考材料校准概念：Anthropic 的 *Introducing the Model Context Protocol*、Anthropic 的 *Beyond permission prompts: making Claude Code more secure and autonomous*、OpenAI 的 *Designing AI agents to resist prompt injection*。会议 transcript 和字幕承担主要内容来源。

2 MCP 的完整定义

2.1 MCP：模型应用连接外部世界的协议层

MCP 是 Model Context Protocol 的缩写。按照 Anthropic 官方介绍，它是一种开放标准，用来把 AI 助手连接到数据源、业务工具和开发环境。更直观地说，MCP 像一组通用插座：模型应用不用为 Slack、GitHub、Gmail、数据库、云服务分别手写一套接线方式，可以通过统一协议发现能力、读取描述、发起调用。

在数学抽象上，可以把一个 Agent 系统写成：

$$a_t = \pi_\theta(g, c_t, \mathcal{T})$$

其中 g 是用户目标， c_t 是当前上下文， $\mathcal{T}$ 是可用工具集合， π_θ 是模型策略。MCP 的作用是把外部工具集合 $\mathcal{T}$ 以结构化方式呈现给模型应用。危险点也正在这里：当工具集合里含有恶意或被污染的描述时，模型策略读到的是会影响行动选择的输入，已经超出单纯文档的性质。

2.2 MCP client、server、tool、description

为了避免术语漂移，本讲义首次完整定义四个核心组件。

术语

定义与安全含义

MCP client	运行在模型应用侧的连接组件。它负责连接 MCP server，获取可用能力，并代表模型应用发起工具调用。它像“插座面板”，让 Agent 能看见并使用外部能力。
MCP server	暴露数据、工具或服务的服务端组件。它把业务系统、开发环境、协作平台或 API 包装成 MCP 可发现、可调用的能力。它像“工具柜”，决定哪些工具被拿出来。
MCP tool	MCP server 暴露给 client 的可调用能力。一个 tool 可以发送消息、查询数据库、读取 issue、检索文件、触发构建或调用业务 API。它像一把带权限的钥匙，能打开真实系统动作。
MCP description	工具的自然语言说明与结构化元数据的一部分，用来告诉模型这个工具做什么、何时使用、参数含义是什么。它像工具旁边的标签；对 Agent 来说，这个标签会参与决策。

演讲中反复强调：LLM 会读取工具名称和 description，然后决定是否调用工具。换句话说，description 在 Agent 系统里已经具有决策输入属性。它会进入模型上下文，成为“可执行意图”的入口。

2.3 credential：凭证是工具调用的能量源

凭证 credential 指 API token、OAuth 授权、服务账号、会话令牌、SSH 凭据等可让工具访问真实系统的权限材料。MCP 工具本身只是路径，凭证才让路径通向真实资产。没有凭证，恶意描述最多影响文本输出；有了凭证，影响范围会触达代码仓库、消息频道、工单系统、云资源和内部数据。

可以用风险表达式概括：

$$R = P(T | A) \times Impact$$

其中 A 是资产， T 是威胁路径。MCP 工具投毒会提高 $P(T | A)$ ，过大的凭证权限会放大 $Impact$ 。如果某个 Agent 同时能读共享频道、写协作工具、访问仓库、调用云 API，那么一个被污染的工具描述就有了横向传播的燃料。

3 从用户请求到工具执行

3.1 正常调用链

正常情况下，MCP 工具调用链可以概括为五步：

1. 用户提出目标，例如“把这段分析发到指定频道”。
2. 模型应用通过 MCP client 查看可用工具。

3. Agent 读取工具名称、description、参数 schema 和上下文。
4. Agent 判断某个工具是否匹配用户目标，并生成结构化调用。
5. MCP client 携带相应凭证调用 MCP server，MCP server 执行业务动作。



图 5: MCP 正常调用链：工具描述参与 Agent 的行动选择，凭证把调用落到真实系统。

这条链路的优势很明显：集成少写重复代码，工具生态更容易扩展，Agent 可以跨多个应用完成任务。安全代价同样清楚：工具边界和描述边界进入了可信计算基的一部分。可信计算基可以理解为系统必须相信的一组组件，一旦这组组件被污染，后续控制都要加倍谨慎。

3.2 description 的特殊性

开发者习惯把 description 看成说明文字。对 Agent 来说，它更接近“策略提示”。工具描述会告诉模型：

- 工具适合什么任务；
- 何时调用；
- 输入参数如何理解；
- 返回值如何解释；
- 某些边界条件下应采取什么行为。

如果 description 被放入模型上下文，它就会与用户请求、系统提示、工具返回、历史对话一起竞争模型注意力。OpenAI 关于 prompt injection 防御的官方文章强调，防御重点应限制被操纵后的影响面，并把 source-sink 关系连起来分析。套到 MCP 场景里，恶意 description 是 source，敏感工具调用、凭证使用、消息发布和数据外传是 sink。

4 工具投毒：把恶意意图藏进工具描述

4.1 定义：工具投毒

工具投毒 tool poisoning 指攻击者操纵工具的自然语言描述、元数据、参数说明或返回内容，使 Agent 在读取这些信息后偏离用户目标或开发者意图，执行攻击者期望的动作。它与 prompt injection 有亲缘关系：二者都利用模型会读取自然语言上下文的事实；差别在于，工具投毒把入口放在工具层，尤其是 description 和工具元数据。

演讲中提到一个关键观察：description 缺少统一安全标准，且常常对用户不可见，开发者也很少逐条审查所有 MCP 工具描述。这个观察很尖锐。系统把 description 放进决策回路，却没有用同等严肃的工程手段治理它。

工具投毒的危险条件

- **隐蔽性**：用户界面通常不展示完整 MCP 连接和工具描述。
- **可变性**：description 可被工具提供方、插件、代理层或供应链环节更新。
- **高权限**：工具调用常携带真实凭证，能读写业务系统。
- **自动化**：Agent 能在多步任务中持续调用工具，降低人工发现概率。
- **缺标准**：元数据安全、描述签名、描述审计、调用隔离尚未形成足够成熟的统一实践。

4.2 描述是“可执行意图入口”

“可执行意图入口”这个说法很重要。它不表示 description 会像二进制程序那样被 CPU 执行；它表示 description 会被模型解释，并影响模型产生下一步动作。对于 Agent 系统，行动通常经过如下映射：

自然语言上下文 → 工具选择 → 参数生成 → 外部执行

恶意描述卡在第一段和第二段之间。它把攻击者意图伪装成工具说明，使模型在“理解工具”时同时吸收隐藏目标。

一个安全的比喻是：传统 API 文档像说明书，读错了最多调用失败；Agent 可读的工具描述像自动驾驶路标，读错了车会真的转向。真正的工程问题在于：系统是否把这个路标当成不可信输入处理。

5 MC Parasite：从单 Agent 影响到 Agent 间传播

5.1 研究重点

MC Parasite 在演讲中被介绍为一个研究与演示工具。它关注的重点超出单次工具调用失败，落在“agent-to-agent worm propagation between the agents”这类传播风险。这里应把“蠕虫”理解为传播模式：一个 Agent 被污染后，通过共享频道或协作工具把影响带给其他 Agent，其他 Agent 读取后继续执行或转发。

5.2 演示面板与实验指标

transcript 中，MC Parasite 的演示面板被描述为可选择频道、攻击场景、模型和提供商，并在右侧做影响分析。这个细节值得保留：它把“某个模型是否中招”变成可比较实

验，防守方可以记录不同模型、不同 provider、不同共享频道在相同投毒场景下的差异。

讲者还提到 9 次测试约 67% 的 infection rate。这个数字不应被外推成通用基准，因为实验环境、模型版本、工具权限和共享频道都会改变结果；它更适合作为评估模板的提示：安全团队需要记录感染率、ASR、敏感输出类别、触发路径、模型/提供商差异和复现条件。涉及 SSH 凭据、环境信息等内容时，讲义只保留为“敏感输出类别”，不展开任何收集或利用步骤。

5.3 代理蠕虫的定义

代理蠕虫 agent worm 指利用 Agent 的自动读取、自动决策、自动工具调用与共享通信能力，在多个 Agent 或多个工作流之间传播恶意指令、污染上下文或触发危险动作的攻击模式。它像传统蠕虫借网络连接复制自己，但传播媒介变成了自然语言上下文、工具描述、共享频道和工具返回。

代理蠕虫需要四类条件同时靠近：

1. 初始污染源，例如被操纵的 MCP 工具 description。
2. 可执行载体，例如能读 description 并调用工具的 Agent。
3. 传播通道，例如 Slack、Jira、Discord、GitHub、工单系统或共享数据库。
4. 可放大权限，例如能读写频道、检索敏感数据、触发自动任务的凭证。

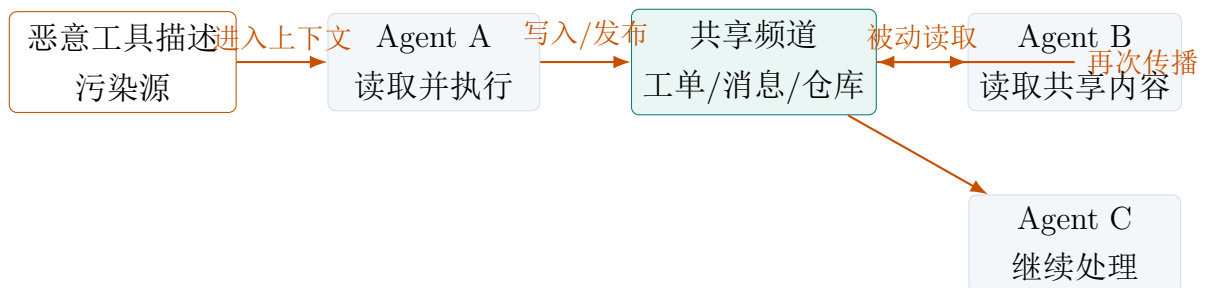


图 6: 代理蠕虫传播模型：工具描述、Agent 执行和共享频道串联后，传播可以脱离持续人工操控。

5.4 “零人工交互”的含义

演讲中有一句关键表达：如果攻击者能操纵第一个 Agent，它就可能执行 agent-to-agent propagation，过程不需要人持续参与。这里的“零人工交互”指传播链上的后续步骤由 Agent 自动读取、自动判断、自动调用工具完成。它并不表示系统完全无人使用；它表示攻击者不必在每一跳手动点击、确认或复制。

这也是防御难点。很多组织把安全假设建在“人会看见异常并中止”上，但 Agent 工作流正在减少人工驻留时间。审批弹窗在高频任务里还会带来 approval fatigue：人被迫反复批准后，判断质量会下降。Anthropic 关于 Claude Code 安全的官方文章也强调，Agent 安全需要文件系统隔离、网络隔离和明确边界，不能把安全全压在权限提示上。

6 凭证权限与共享频道

6.1 凭证权限如何放大影响

MCP 工具常常连接真实服务：协作频道、代码仓库、邮件、云 API、工单系统、CRM、数据库。每个连接都需要凭证。凭证范围决定攻击后的影响半径：

- 只读凭证可能泄露信息、帮助侦察或收集上下文。
- 写权限凭证可能发布污染内容、修改工单、创建 issue、触发自动任务。
- 管理员凭证可能改变配置、授权更多工具、扩大访问范围。
- 跨系统凭证组合会形成“权限拼图”，单个工具看似有限，组合后能影响整个工作流。

对于 Agent，凭证应避开“这个人平时能做什么”的粗放授权，改按“这个自动化任务完成目标所需的最小动作”授予。最小权限要落成工程手段，用来把攻击后的损害面积压小。

6.2 共享频道为何危险

共享频道在组织里很正常：团队消息、工单评论、PR 讨论、知识库页面、自动化报告。对 Agent 系统来说，它们同时具备三种属性：

- **输入源**：Agent 会读取它们作为上下文。
- **输出口**：Agent 会把结果写回它们。
- **传播面**：其他 Agent、自动化流程和人员会继续读取。

这三种属性叠在一起，频道就像组织内部的传送带。正常信息靠它流动，污染信息也会靠它流动。OpenAI 的 prompt injection 防御建议强调 source-sink 分析，在这里可以扩展为：

$$Source_{description} \rightarrow Sink_{tool\ call} \rightarrow Source_{shared\ channel} \rightarrow Sink_{next\ agent}$$

也就是说，一次输出可能变成下一次输入。防御如果只看单次调用，就会漏掉跨 Agent 传播。

7 从 MC Parasite 提炼出的 kill chain

为避免提供可直接滥用的操作步骤，本节只给出防御建模所需的抽象链路。

阶段	攻击者目标	防御者应观测的信号
----	-------	-----------

工具入口	让 Agent 读取被污染的工具描述或元数据	新增 MCP server、description 变更、工具 schema 异常、来源不明工具接入
执行决策	诱导 Agent 选择特定工具或生成危险参数	工具调用与用户目标不匹配、参数含敏感检索范围、调用频率异常
凭证使用	借工具凭证访问真实系统	凭证跨系统使用、越权读写、服务账号行为偏离基线
共享传播	将污染内容写入频道、工单、仓库或知识库	Agent 输出包含指令性文本、隐藏上下文、可被后续 Agent 读取的异常内容
横向扩散	让后续 Agent 继续读取并执行	多 Agent 在短时间内重复相似调用、跨频道相似文本扩散、自动任务链式触发

这张表的重点是防守视角。安全团队需要把 MCP 工具描述、工具调用、凭证使用、共享频道写入和后续 Agent 读取放在同一条审计链里。只记录 API 调用，不记录 description 版本；只记录消息写入，不记录哪个 Agent 读过；只记录最终错误，不记录中间工具选择，这些都会让复盘变得残缺。

8 防御建议：把工具描述当作不可信输入

8.1 一、工具白名单与来源治理

组织应维护 MCP server 与 tool 的白名单。白名单不只是工具名称列表，还应包含来源、负责人、版本、权限范围、描述哈希、参数 schema、可访问资产和变更审批记录。新增工具进入生产 Agent 前，需要经过安全评审和最小可用性测试。

对于第三方 MCP server，建议按供应链组件管理。组件越靠近凭证和业务系统，审查越严格。工具描述和参数 schema 也应纳入版本管理，不能让“文字说明”游离在代码审查之外。

8.2 二、description 审计与签名

description 应作为安全敏感元数据处理。可执行措施包括：

- 对工具 description 做版本化存储和差异审查。
- 对生产环境工具元数据做签名或哈希校验。
- 检测描述中与工具职责无关的指令性文本。

- 将用户可见摘要与模型可读 description 分离，并记录完整内容。
 - 对 description 变更触发自动化测试，验证 Agent 不会越过预期边界。
- 直白地说，工具描述已经进入执行链，继续把它当普通 README 管，风险会越来越高。

8.3 三、最小权限与短寿命凭证

工具凭证应遵循最小权限、短寿命、按任务分区。具体原则如下：

- 不给通用 Agent 配置长期高权限 token。
 - 读写权限分离，敏感写操作单独授权。
 - 按工具、频道、仓库、环境划分凭证作用域。
 - 对高风险动作使用短期令牌和二次确认。
 - 凭证使用日志必须能关联到用户目标、Agent 决策和工具 description 版本。
- 安全控制的目标是让 Agent 保持可用，同时让被操纵后的影响撞到小范围缓冲墙。

8.4 四、执行隔离与网络隔离

Anthropic 关于 Claude Code 安全的官方文章指出，沙箱与隔离可以让 Agent 在边界内更自主，在越界时触发审批。MCP 场景同样需要隔离：

- 文件系统隔离：限制工具能读写的目录、仓库和临时文件。
- 网络隔离：限制 MCP server 可访问的主机、域名和端口。
- 数据隔离：把生产数据、测试数据、公开数据分开。
- 身份隔离：不同 Agent、不同任务、不同环境使用不同身份。
- 执行隔离：高风险工具运行在可回滚、可观测、可限制资源的环境里。

8.5 五、source-sink 分析与敏感动作门禁

OpenAI 的 Agent prompt injection 防御建议强调限制被操纵后的影响面。落到 MCP，可以建立 source-sink 清单：

Source：不可信输入源	Sink：敏感动作汇点
MCP tool description	使用高权限凭证调用外部 API
工具返回内容	写入共享频道、工单、PR 评论、知识库

网页、邮件、文档内容	读取私有仓库、导出数据、触发部署
其他 Agent 的输出	修改配置、创建授权、批量发送消息

凡是从 source 流向 sink 的路径，都应有策略约束、日志和必要的人类确认。确认不应是“是否允许这个工具运行”的笼统问题，而应展示：输入来源、将执行的动作、目标资产、凭证范围、预期输出位置。

9 组织落地清单

9.1 开发者清单

- 为每个 MCP tool 写清楚单一职责，避免 description 承担过多策略逻辑。
- 工具 schema 尽量收紧参数类型和取值范围，减少自由文本参数。
- 把工具 description 纳入代码审查、CI 检测和发布签名。
- 对 Agent 调用工具的决策过程做 trace，保存用户目标、候选工具、最终参数。
- 对共享频道写入设置内容检查，防止 Agent 输出变成下游 Agent 的隐藏指令。

9.2 安全团队清单

- 建立 MCP 资产清单：server、client、tool、description、credential、owner。
- 对高风险工具做红队评估，重点检查工具投毒、凭证滥用、共享频道传播。
- 监控 description 变更、异常工具调用和跨 Agent 重复传播模式。
- 设计事故响应流程：禁用工具、吊销凭证、隔离频道、回滚 description、复盘 trace。
- 把 MCP 纳入供应链安全、身份权限管理和数据防泄漏策略。

9.3 管理者清单

- 明确 MCP 工具接入的责任人和审批门槛。
- 不用“提高效率”压过权限边界和审计要求。
- 为 Agent 安全投入隔离、监控、测试和复盘资源。
- 要求供应商说明 MCP 元数据标准、工具描述治理和安全更新机制。
- 把零人工交互场景列为高风险自动化，单独做上线评审。

10 课堂总结

MCP 的价值在于让 Agent 更容易连接真实工具。风险也正来自这里：连接越顺滑，错误意图进入执行链的速度越快。MC Parasite 的研究提醒安全团队，工具 description、凭证权限、共享频道和多 Agent 自动化不能分开看。它们像四块齿轮，单独看每块都合理，咬合起来就可能形成传播机器。

本主题最该带走的判断有五个：

1. MCP description 是 Agent 决策输入，必须按不可信输入治理。
2. 工具投毒的危险点在于隐蔽、自动化和凭证放大。
3. 代理蠕虫依靠共享频道和多 Agent 工作流传播，后续跳数可能脱离攻击者手动控制。
4. 防御不能停在权限弹窗，应覆盖白名单、审计、最小权限、隔离和 source-sink 分析。
5. 组织要为 MCP 建立元数据标准和安全标准，否则 Agent 生态会把“说明文字”变成新的供应链入口。

官方参考

- Anthropic, *Introducing the Model Context Protocol*. <https://www.anthropic.com/news/model-context-protocol>。用于校准 MCP 的协议定位、client/server 连接关系与开放标准表述。
- Anthropic, *Beyond permission prompts: making Claude Code more secure and autonomous*. <https://www.anthropic.com/engineering/claude-code-sandboxing>。用于校准 Agent 安全中的沙箱、隔离与权限提示疲劳问题。
- OpenAI, *Designing AI agents to resist prompt injection*. <https://openai.com/index/designing-agents-to-resist-prompt-injection/>。用于校准 source-sink 分析、敏感动作约束和降低被操纵后影响面的防御思路。